

TP2 : VHDL Séquentiel

Jaime GOMES-PARENTE et Robin BRICHE

Introduction

Ce rapport présente les réponses et résultats du TP numéro 2. L'objectif principal de ce TP est de concevoir, simuler et implémenter des circuits nécessitant la mémorisation de leur état, tels que des compteurs et des machines à états.

Au cours de ce TP, nous avons d'abord étudié l'impact de la conception VHDL sur la synthèse matérielle en comparant un reset asynchrone et un reset synchrone sur un compteur. Ensuite, nous avons modélisé et implémenté deux versions d'un détecteur pour la séquence 11010.

Exercice 1 : Compteur synchrone 2 bits avec reset asynchrone

1. Modélisez ce compteur en VHDL.

Le cahier des charges demande un compteur sur 2 bits qui s'incrémente sur le front montant d'une horloge fait avec le bouton droit et qui possède une remise à zéro asynchrone faite avec le bouton gauche. Pour obtenir ce comportement asynchrone, le signal de reset doit être placé dans la liste de sensibilité du process, et sa condition doit être évaluée indépendamment du front d'horloge. On propose donc le code VHDL ci-dessous pour répondre à ce question :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL; -- Permet de faire l'addition '+' directement sur du std_logic_vector

) entity ex01 is
    Port (
        BTN_D : in  STD_LOGIC;           -- Horloge
        BTN_G : in  STD_LOGIC;           -- Reset asynchrone
        LED_1_0 : out STD_LOGIC_VECTOR (1 downto 0) -- Compteur
    );
) end ex01;

) architecture Behavioral of ex01 is
    signal count : STD_LOGIC_VECTOR(1 downto 0) := "00";
    begin
        -- Le process est obligatoire pour réagir sur un front d'horloge
        ) process(BTN_D, BTN_G)
            begin
                ) if BTN_G = '1' then
                    count <= "00";
                    elsif BTN_D'event and BTN_D = '1' then -- Syntaxe classique/débutant pour le front montant
                        count <= count + 1;
                    ) end if;
                ) end process;

                LED_1_0 <= count;
            ) end Behavioral;
```

Figure 1.1 : Code VHDL question1

2. Simulez le circuit et observez son fonctionnement.

La simulation comportementale met en évidence l'asynchronisme de la remise à zéro. Dès que le signal `BTN_G` passe à l'état haut, la sortie s'annule instantanément sans attendre le prochain front montant de `BTN_D`.

On voit bien que sans bouton gauche qui monte à 0, le compteur fait des boucles de 0 à 3 et lorsque le bouton gauche s'active, on a le compteur qui se reinitialise dès que celui-ci vaut 1.

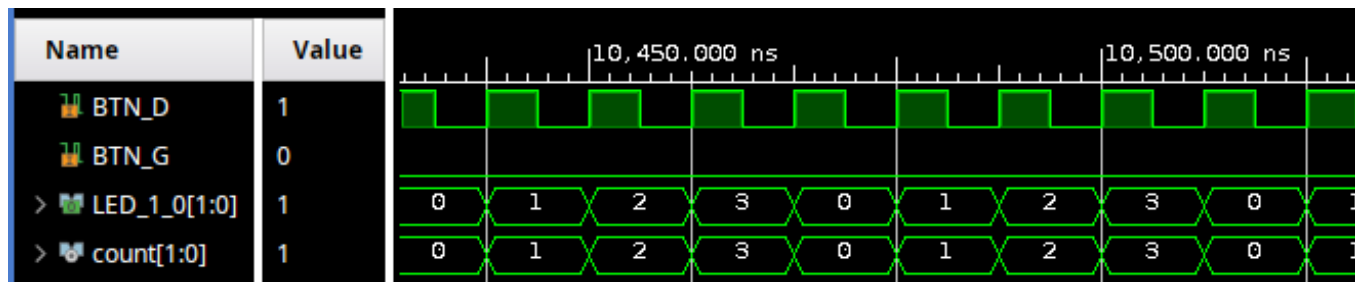


Figure 1.2 : Simulation avec variation de `BTN_D` uniquement

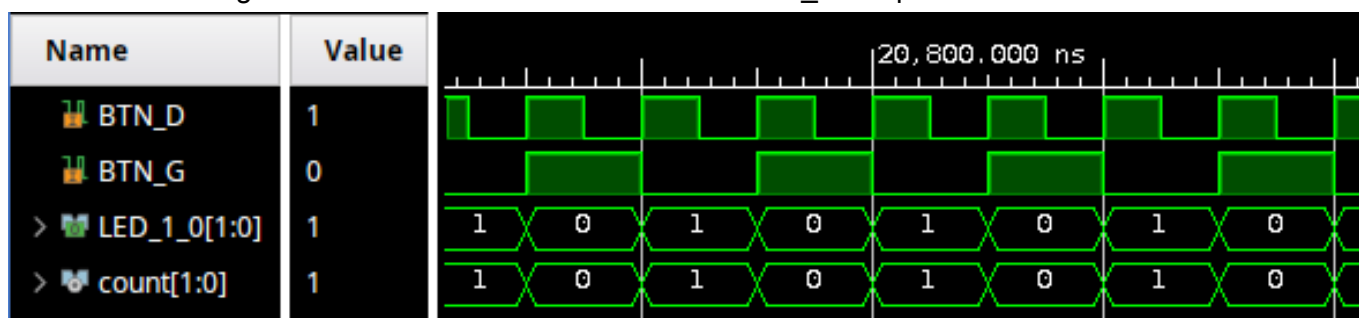


Figure 1.3: Simulation avec variation de `BTN_D` et `BTN_G`

3. Lancez l'implémentation. Une fois celle-ci terminée, ouvrez le schéma « Implémentation ». Expliquez le fonctionnement du compteur à partir du schéma. Quels éléments matérialisent la valeur du compteur dans ce schéma ?

À partir du schéma d'implémentation, on constate que la valeur du compteur est matérialisée par des bascules D matérielles, spécifiquement de type FDCE (Flip-Flop with Clock Enable and Asynchronous Clear). Le fonctionnement est le suivant : l'entrée d'horloge C est reliée à `BTN_D`. Le signal de reset `BTN_G` est directement routé sur la broche CLR (Clear) des bascules. Cette broche a pour effet de forcer la sortie Q à 0 de manière asynchrone (indépendamment de l'horloge), ce qui correspond parfaitement à la description VHDL.

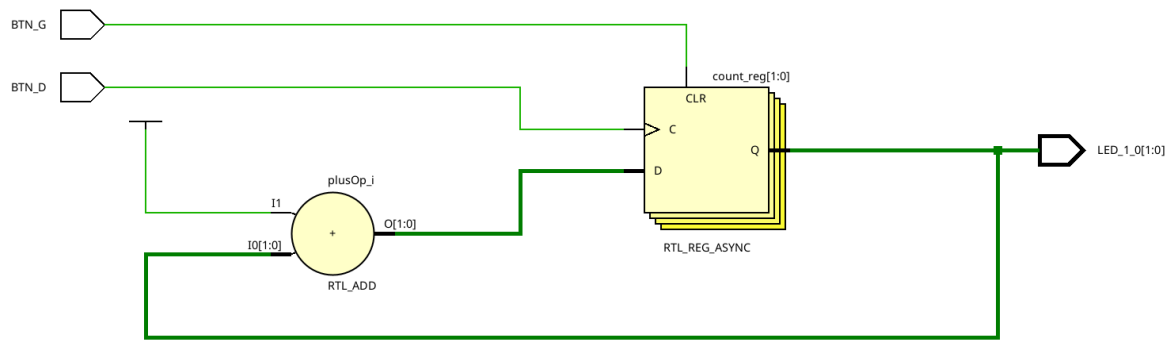


Figure 1.4 : Schéma avant synthétisation

count[0]_i_1			count[1]_i_1		
I0	O=!I0		I1	I0	O=I0 & !I1 + !I0 & I1
0	1		0	0	0
1	0		0	1	1
			1	0	1
			1	1	0

Figure 1.5 : table de vérité du schéma précédent

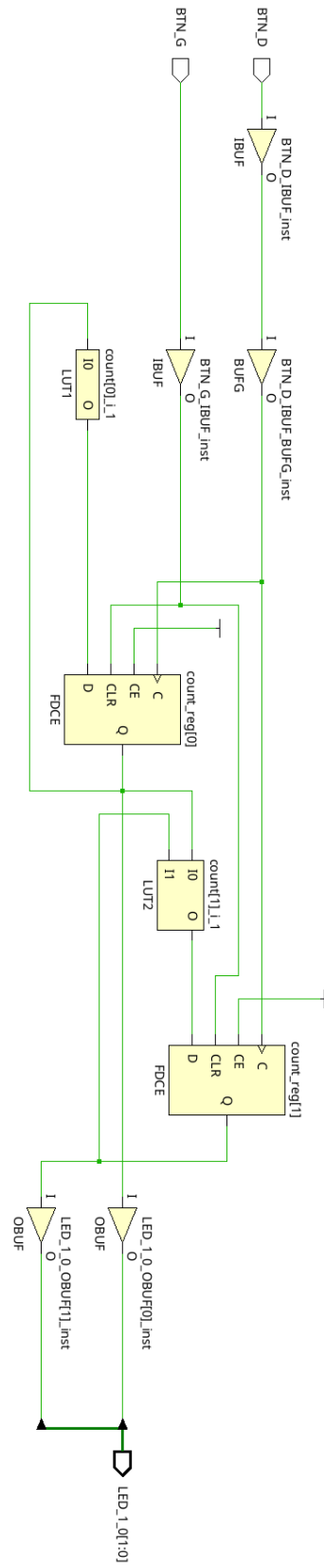
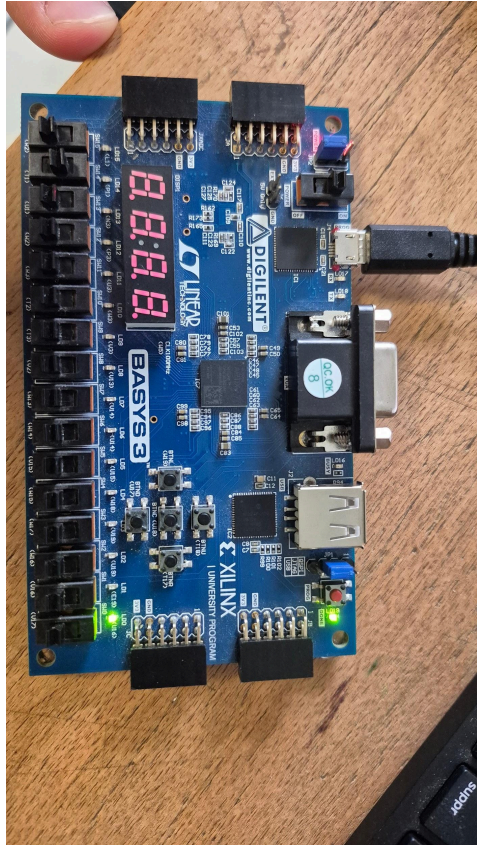


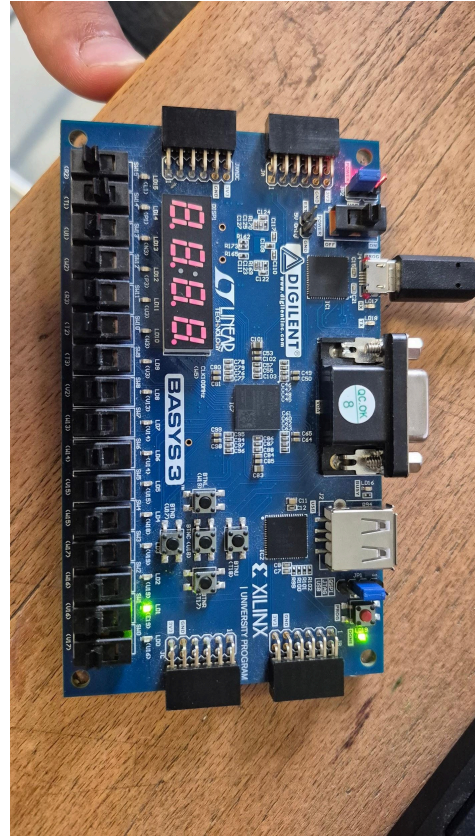
Figure 1.6 : schéma après implémentation

4. Programmez le FPGA et constatez le fonctionnement de votre circuit.

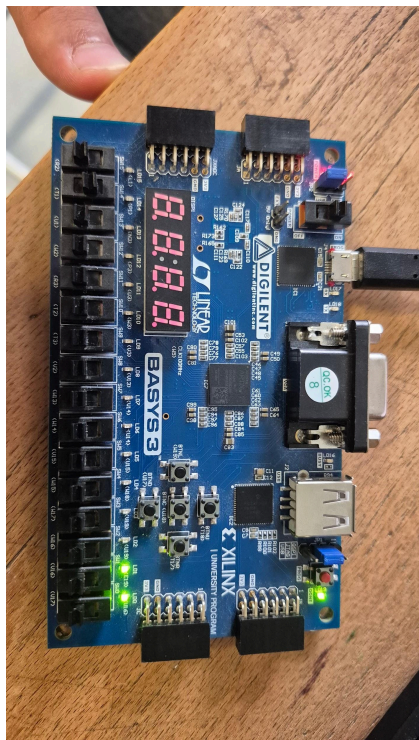
On voit sur les photos ci-dessous que si on ne touche qu'au bouton droit, le compteur s'incrémente et les leds s'allument petit à petit. Si le bouton gauche est appuyé, on retourne au stade où aucune led n'est allumée.



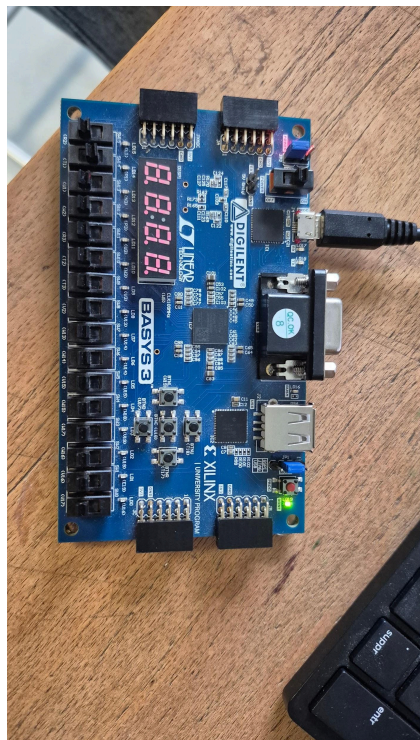
1er appui : Led 0 allumée



2nd appui : Led 1 allumée



3eme appui : Led 0 et 1 allumée



4eme appui : retour à 0 led allumée

Figure 1.7 : Tableau des 4 phases possibles pour les leds.

Exercice 2 : Compteur synchrone 2 bits avec reset synchrone

1. Reprenez le compteur de l'exercice 1 et rendez le signal reset synchrone.

Dans cet exercice, la réinitialisation doit être synchrone, c'est-à-dire qu'elle ne doit être prise en compte qu'au moment d'un front d'horloge. Pour traduire cela en VHDL, le signal BTN_G doit être retiré de la liste de sensibilité du process. La vérification de l'état de BTN_G doit s'effectuer exclusivement à l'intérieur de la condition `rising_edge(BTN_D)`.

On propose donc ce code VHDL pour le modéliser :

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity exo2 is
    Port (
        BTN_D : in  STD_LOGIC;           -- Horloge (front montant)
        BTN_G : in  STD_LOGIC;           -- Reset synchrone (actif à '1')
        LED_1_0 : out STD_LOGIC_VECTOR (1 downto 0) -- Valeur du compteur
    );
end exo2;

architecture Behavioral of exo2 is
    signal count : STD_LOGIC_VECTOR(1 downto 0) := "00";
begin

    -- Remarque : BTN_G n'est PAS dans la liste de sensibilité
    process(BTN_D)
    begin
        if rising_edge(BTN_D) then -- ou (BTN_D'event and BTN_D = '1')
            if BTN_G = '1' then
                count <= "00";      -- Reset appliqué uniquement sur front d'horloge
            else
                count <= count + 1; -- Incrémentation si pas de reset
            end if;
        end if;
    end process;

    LED_1_0 <= count;
end Behavioral;
```

Figure 2.1 : Code VHDL

Contrairement à l'exercice 1, le signal de réinitialisation BTN_G n'est pas inclus dans la liste de sensibilité du process. De plus, la condition de test du reset `if BTN_G = '1'` est imbriquée à l'intérieur de la condition de front d'horloge `if rising_edge(BTN_D)`. Par conséquent, le système analyse l'état du bouton de reset uniquement au moment exact où survient le front montant de l'horloge.

2. Simulez le circuit et observez son fonctionnement.

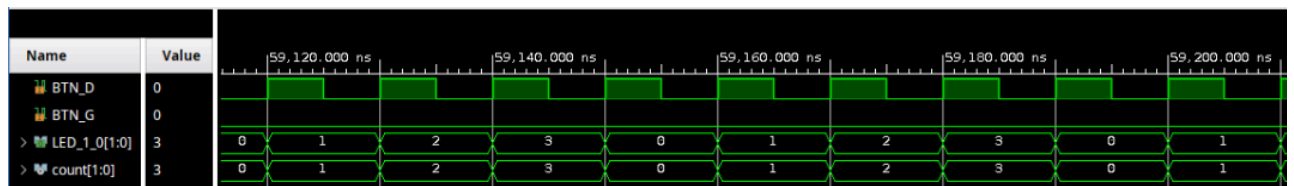


Figure 2.2 Simulation avec BTN_D = 0

Première simulation (Comptage normal, BTN_G = '0') : Sur le premier chronogramme, lorsque le reset BTN_G est maintenu à 0, on observe que le signal count s'incrémente régulièrement de 0 à 3 à chaque front montant de l'horloge BTN_D (0 -> 1 -> 2 -> 3 -> 0...).



Figure 2.3 Simulation avec BTN_D changeant

On peut observer sur la simulation que le comportement est bien devenu synchrone. Sur notre chronogramme, on remarque qu'au niveau de la graduation 2 010 ns, le signal d'horloge BTN_D effectue un front montant alors que le signal de reset BTN_G est actif à 1. C'est exactement à ce moment précis sur le front d'horloge que le signal count bascule à 0. Le reset s'exécute donc bien de manière synchronisée.

3. Lancez l'implémentation. Une fois celle-ci terminée, ouvrez le schéma « Implémentation ». En le comparant au schéma du compteur précédent, expliquez ce qui fait que le reset est synchrone dans cet exercice et asynchrone dans l'exercice précédent.

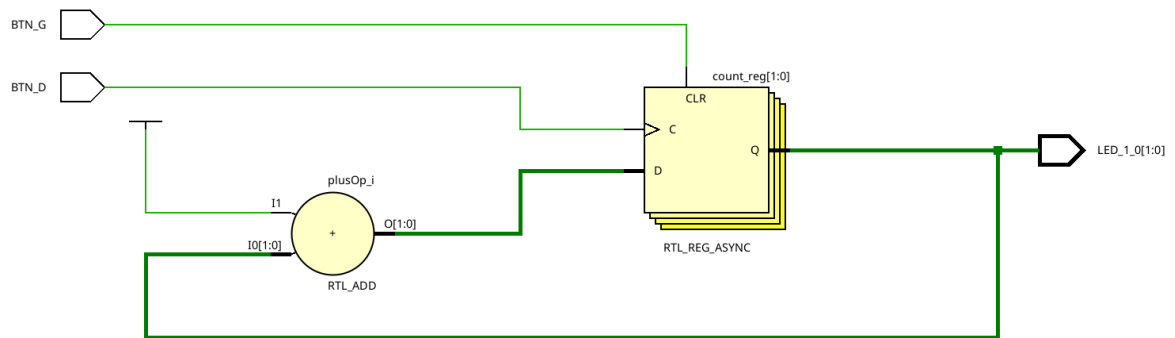


Figure 2.4 schéma avant synthétisation

count[0]_i_1			count[1]_i_1			
I1	I0	O=!I0 & !I1	I2	I1	I0	O=I0 & !I1 & !I2 + !I0 & I1 & !I2
0	0	1	0	0	0	0
0	1	0	0	0	1	1
1	0	0	0	1	0	1
1	1	0	0	1	1	0
			1	0	0	0
			1	0	1	0
			1	1	0	0
			1	1	1	0

Figure 2.5 Table de vérité du schéma d'implémentation

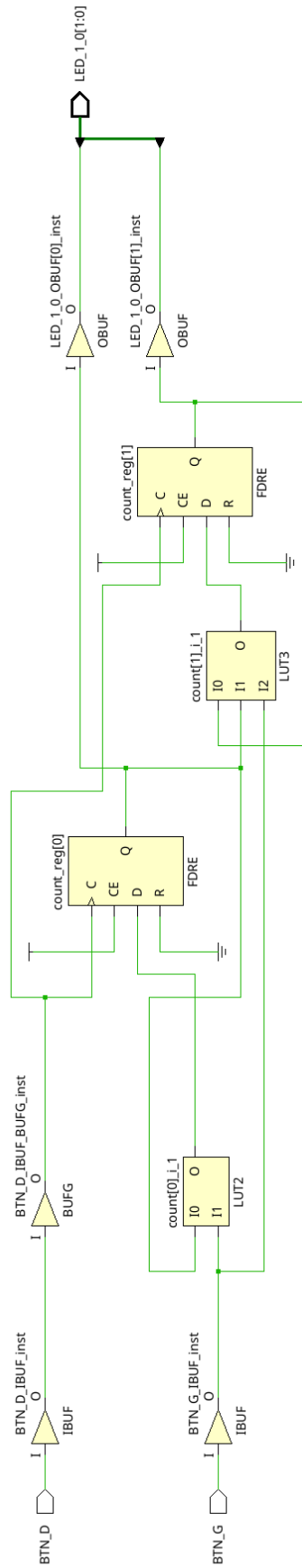


Figure 2.6 Schéma d'implémentation

En observant le schéma d'implémentation, on remarque que le synthétiseur a utilisé des blocs de type FDRE, c'est-à-dire des bascules D avec Clock Enable et Synchronous Reset. Contrairement à l'exercice précédent où le reset attaquait une entrée asynchrone, le signal de reset est ici géré par l'entrée R de la bascule, garantissant une remise à zéro matérielle purement synchrone.

4. Programmez le FPGA et constatez le fonctionnement de votre circuit.

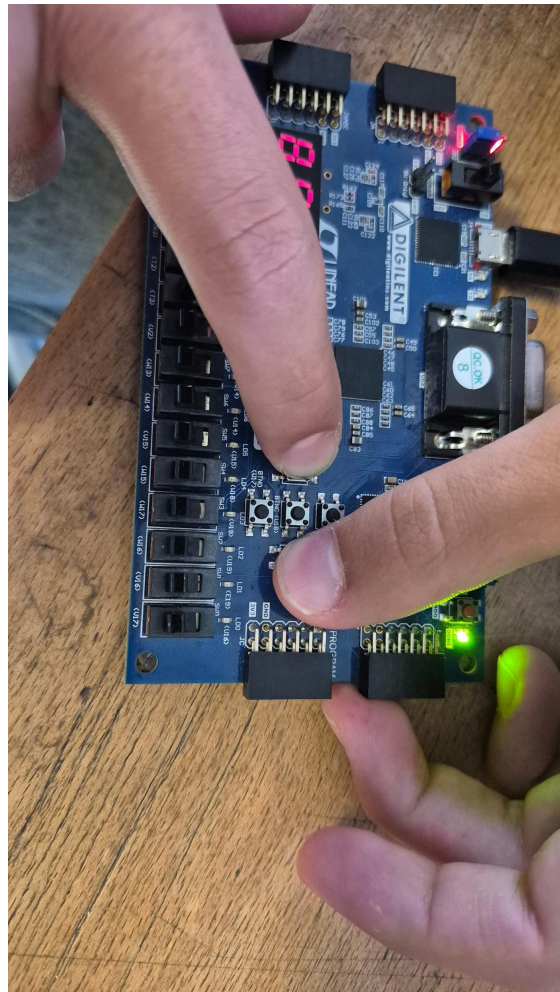


Figure 2.7 Photo montrant le reset synchrone

Lors de la programmation et des tests sur la carte FPGA, le fonctionnement physique confirme notre modèle : un appui isolé sur le bouton de reset (gauche) ne réinitialise pas le compteur. Pour que la remise à zéro s'effectue, il faut maintenir le bouton de reset enfoncé et actionner simultanément le bouton d'horloge. L'action de reset attend donc bien le coup d'horloge pour être validée physiquement.

Exercice 3 : Détecteur de code

Dans cet exercice, nous avons pour but de réaliser un récepteur série synchrone capable de détecter la séquence 11010 sur l'entrée SW_0, et d'activer la LED_0 lorsque la séquence est entrée correctement, tout en réinitialisant complètement l'entrée de l'utilisateur à la moindre erreur

1. Modélisez ce détecteur sous forme d'une machine à état

Pour formaliser la reconnaissance de la séquence 11010 et les transitions vers l'état initial en cas d'erreur, on établit le graphe d'état.

(Site utilisé pour la modélisation : <https://machines.brunet-zamansky.fr/automate/>)

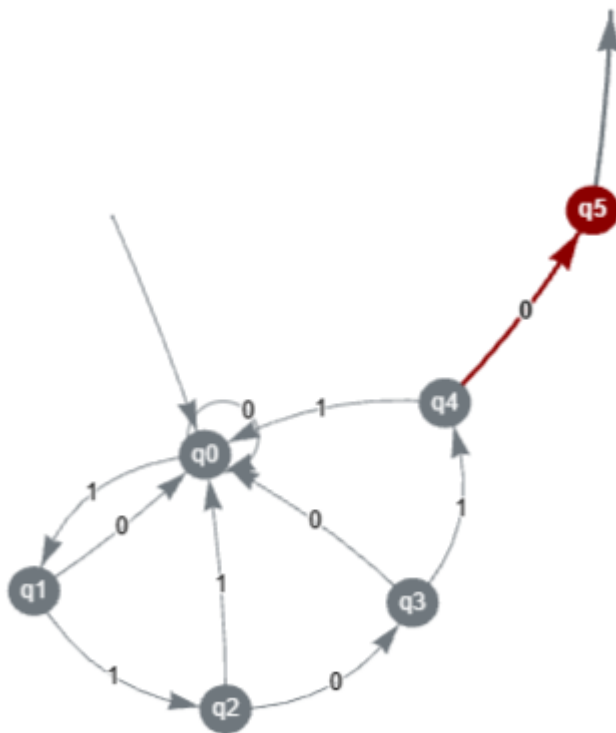


Figure 3.1 : Machine d'état de l'exercice 3

2. Implémentez cette machine à états en VHDL.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity exo3 is
    Port (
        SW_0 : in  STD_LOGIC;
        BTN_D : in  STD_LOGIC;
        LED_0 : out STD_LOGIC
    );
end entity exo3;
```

```

architecture detecteur of exo3 is
begin
    process(BTN_D)
        variable etat : integer range 0 to 5 := 0;
    begin
        if rising_edge(BTN_D) then
            case etat is
                when 0 =>
                    if SW_0 = '1' then
                        etat := 1;
                    end if;

                    when 1 =>
                        if SW_0 = '1' then
                            etat := 2;
                        else
                            etat := 0;
                        end if;

                    when 2 =>
                        if SW_0 = '0' then
                            etat := 3;
                        else
                            etat := 0;
                        end if;

                    when 3 =>
                        if SW_0 = '1' then
                            etat := 4;
                        else
                            etat := 0;
                        end if;

                    when 4 =>
                        if SW_0 = '0' then
                            etat := 5;
                        else
                            etat := 0;
                        end if;

                    when others =>
                        etat := 0;
            end case;

            if etat = 5 then
                LED_0 <= '1';
            else
                LED_0 <= '0';
            end if;

        end if;
    end process;
end architecture detecteur;

```

Figure 3.2 : Code VHDL du detecteur de code

3. Simulez le détecteur et observez son fonctionnement.

On crée deux clocks binaires avec des périodes différentes pour les deux entrées, et on lance la simulation. On remarque que la LED_0 reste éteinte la plus part du temps, ce qui est logique car les clocks produisent des combinaisons aléatoires. On retrouve cependant que les 5 fronts montants de SW_0 qui précèdent le passage à 1 de LED_0 forment la combinaison 11010, et que la led s'éteint dès le front montant suivant de clock.



Figure 3.3 : Simulation du détecteur de code

4. Au moyen du schéma « Implémentation », déterminez le nombre de bits d'état utilisés par le détecteur. Est-ce cohérent avec la représentation de l'état de votre modèle VHDL ?

Sur le schéma initial, la variable entière (de 0 à 5) est codée sur 3 bits (car $2^3 = 8 > 6$). Or, dans le schéma après implémentation, le logiciel a choisi d'utiliser 6 bits, avec un bit par état. Ainsi, ce n'est pas une traduction directe du code VHDL que nous avons écrit, mais une optimisation effectuée par Vivado pour limiter le temps d'exécution.

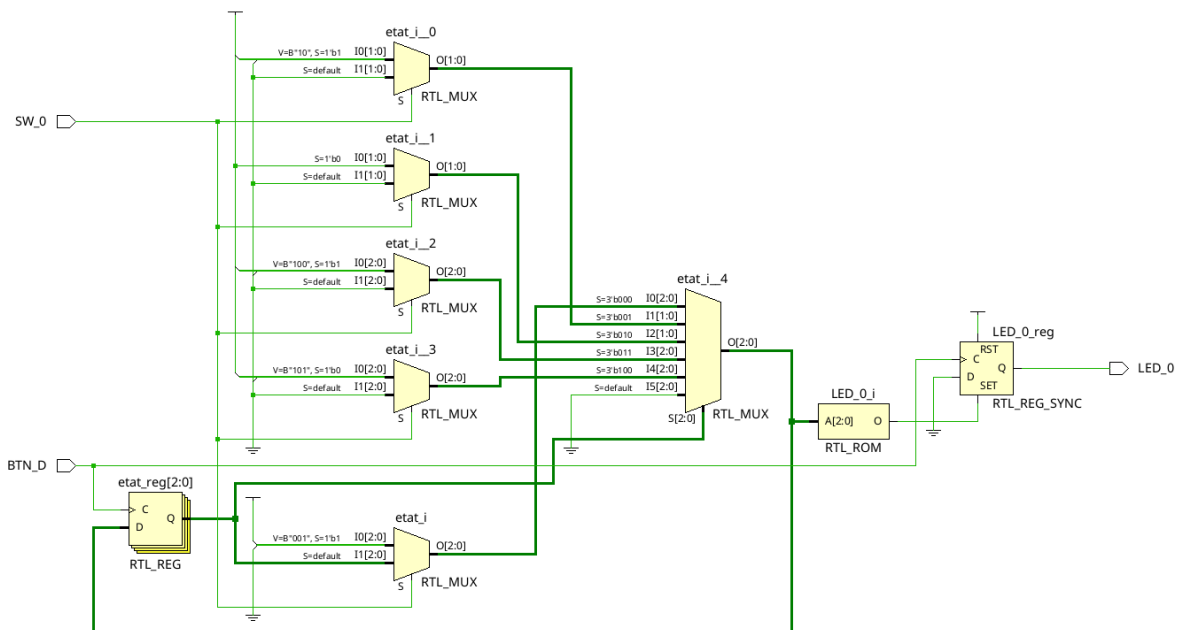


Figure 3.4 : Schéma du détecteur de code avant synthetisation

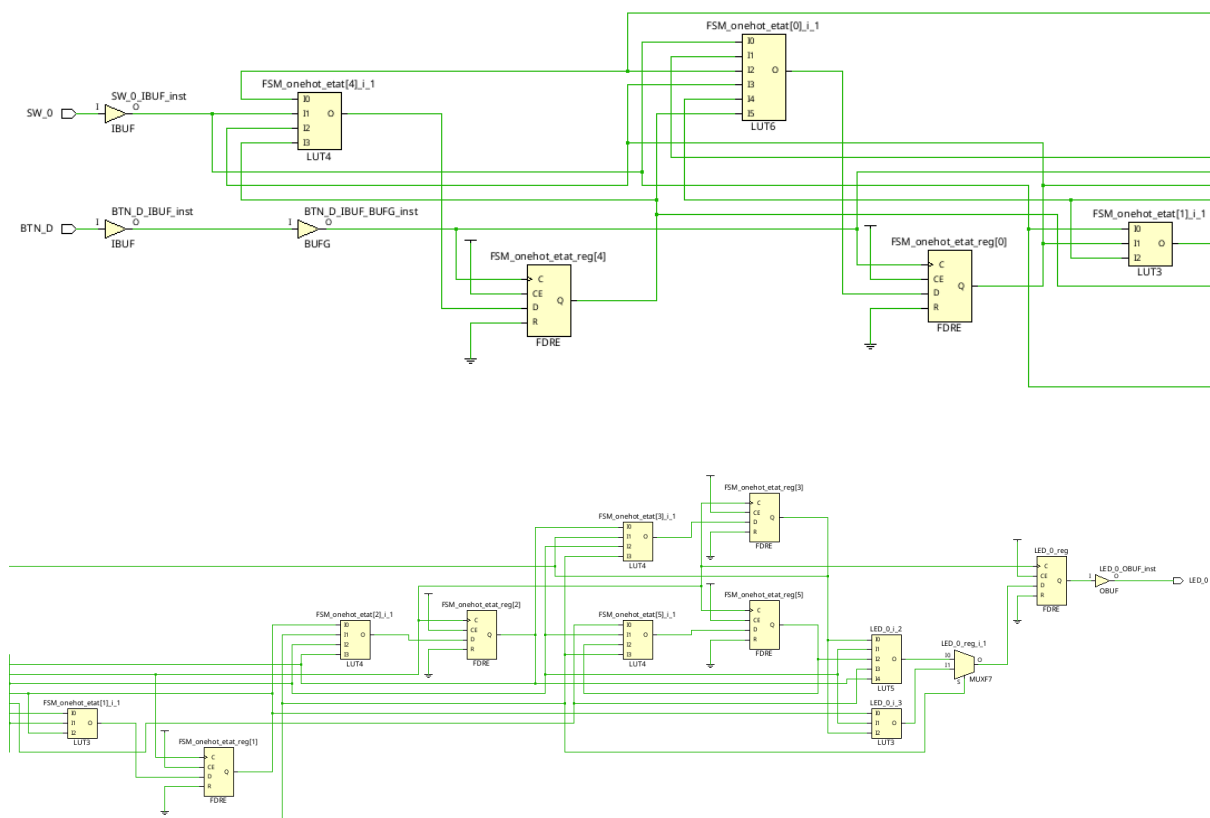


Figure 3.5 : Schéma du détecteur de code après synthétisation

FSM_onehot_etat[4]_i_1				
I3	I2	I1	I0	O=I0 & I1 + I1 & I2 & I3
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

FSM_onehot_etat[1]_i_1			
I2	I1	I0	O=I0 & I1 + I1 & I2
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

FSM_onehot_etat[0]_i_1							
I5	I4	I3	I2	I1	I0	O=I0 & I1 & !I5 + I2 & !I4 & I5 + !I2 & I3 & I0	
0	0	0	0	0	0	1	
0	0	0	0	0	1	1	
0	0	0	0	1	0	0	
0	0	0	0	1	1	1	
0	0	0	1	0	0	1	
0	0	0	1	0	1	0	
0	0	0	1	1	0	1	
0	0	0	1	1	1	1	
0	0	1	0	0	0	1	
0	0	1	0	0	1	0	
0	0	1	0	1	0	1	
0	0	1	0	1	1	1	
0	0	1	1	0	0	1	
0	0	1	1	0	1	0	
0	0	1	1	1	0	1	
0	0	1	1	1	1	1	
0	1	0	0	0	0	1	
0	1	0	0	0	1	0	
0	1	0	0	1	0	1	
0	1	0	0	1	1	1	
0	1	0	1	0	0	1	
0	1	0	1	0	1	0	
0	1	0	1	1	0	1	
0	1	0	1	1	1	1	
0	1	1	0	0	0	1	
0	1	1	0	0	1	0	
0	1	1	0	1	0	1	
0	1	1	0	1	1	1	
0	1	1	1	0	0	1	
0	1	1	1	0	1	0	
0	1	1	1	1	0	1	
0	1	1	1	1	1	1	

FSM_onehot_etat[0]_i_1							
I5	I4	I3	I2	I1	I0	O=I0 & I1 & !I5 + I2 & !I4 & I5 + !I2 & I3 & I0	
1	0	0	0	0	0	0	
1	0	0	0	0	1	1	
1	0	0	0	1	0	0	
1	0	0	0	1	1	1	
1	0	0	1	0	0	1	
1	0	0	1	0	1	1	
1	0	0	1	1	0	1	
1	0	0	1	1	1	1	
1	0	1	0	0	0	1	
1	0	1	0	0	1	1	
1	0	1	0	1	0	1	
1	0	1	0	1	1	1	
1	0	1	1	0	0	1	
1	0	1	1	0	1	1	
1	0	1	1	1	0	1	
1	0	1	1	1	1	1	
1	1	0	0	0	0	1	
1	1	0	0	0	1	1	
1	1	0	0	1	0	1	
1	1	0	0	1	1	1	
1	1	0	1	0	0	1	
1	1	0	1	0	1	1	
1	1	0	1	1	0	1	
1	1	0	1	1	1	1	
1	1	1	0	0	0	1	
1	1	1	0	0	1	1	
1	1	1	0	1	0	1	
1	1	1	0	1	1	1	
1	1	1	1	0	0	1	
1	1	1	1	0	1	1	
1	1	1	1	1	0	1	
1	1	1	1	1	1	1	

LED_0_i_2					
I4	I3	I2	I1	I0	O=!!I0 & I1 & I2 & !I4 + !I1 & I3 & !I4 + !I0 & I3 & !I1
0	0	0	0	0	0
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	0	1	0
0	0	1	1	0	1
0	0	1	1	1	0
0	1	0	0	0	1
0	1	0	0	1	1
0	1	0	1	0	1
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	0
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	0	1	0
1	1	0	1	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	0

Edit LUT Equation...

il Properties Power Nets Cell Pins Truth Table

Figures 3.6 - 3.10 : Tables de vérité du schéma d'implémentation

5. Quelle est l'équation logique des bits d'états 3 utilisée pour générer la sortie ?

La LED LED_0 ne s'allume uniquement lorsque la machine atteint son dernier état q5; on peut traduire ça par l'équation:

$$LEd_0 = FSM_oneshot_etat[5]$$

6. Programmez le FPGA et vérifiez le fonctionnement du détecteur.

Afin de vérifier le fonctionnement du détecteur, nous venons configurer l'entrée souhaitée via le switch SW_0 puis nous validons chaque bit en appuyant sur le bouton poussoir de droite BTN_D, qui sert ici d'horloge. On saisit ensuite la séquence bit par bit: 1 (SW_0 vers le haut puis BTN_D), 1 à nouveau, 0 (SW_0 vers le bas puis BTN_D), 1, 0. Dès le dernier front d'horloge validant le dernier bit, LED_0 s'allume bien, montrant que le programme reconnaît bien la séquence 11010. En cas d'erreur, LED_0 reste éteinte et la machine retourne à l'état initial ; il faut recommencer pour valider la séquence.

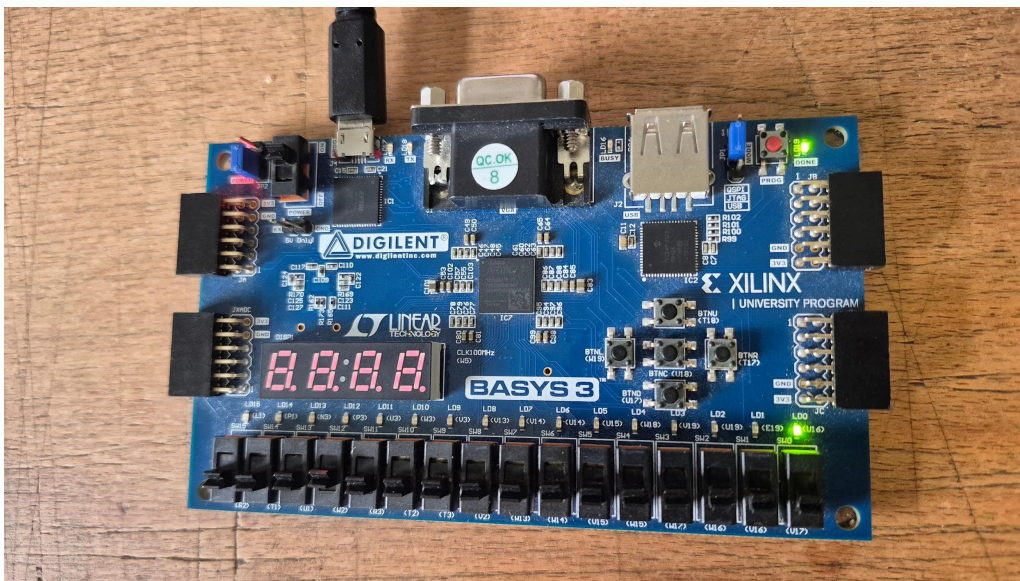


Figure 3.11 : FPGA après avoir entré la séquence correctement

Exercice 4 : Détecteur de code avec fausses entrées négligées

Dans ce dernier exercice, nous allons reprendre le concept de l'exercice 3 tout en assouplissant les conditions pour activer la LED_0. Désormais, une erreur ne remet pas à 0 l'entrée de la séquence.

1. Reprenez l'exercice 3 de manière à négliger les fausses entrées.

En modélisant la machine d'état, on remarque qu'elle est bien différente de celle de l'exercice 3; En effet, les erreurs nous maintiennent sur l'état actuel au lieu de nous renvoyer vers l'état initial q0.

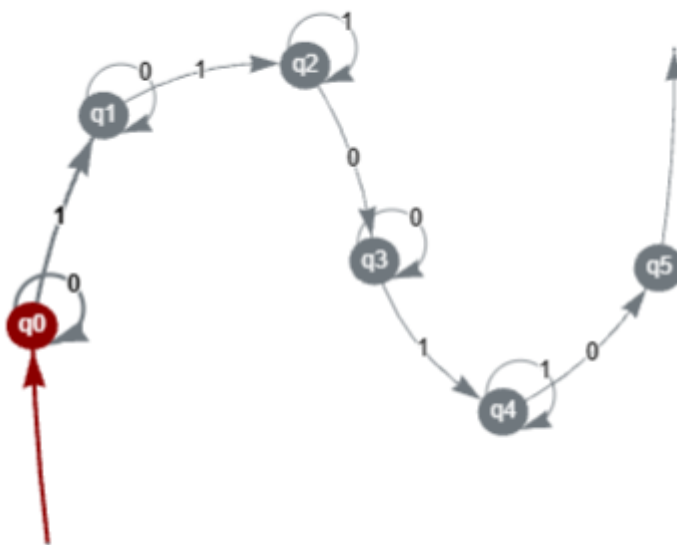


Figure 4.1 : Machine à états du détecteur de code assoupli

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity exo4 is
    Port (
        SW_0 : in  STD_LOGIC;
        BTN_D : in  STD_LOGIC;
        LED_0 : out STD_LOGIC
    );
end entity exo4;

architecture Behavioral of exo4 is
begin
    process(BTN_D)
        variable etat : integer range 0 to 5 := 0;
    begin
        if rising_edge(BTN_D) then
            case etat is
                when 0 =>
                    if SW_0 = '1' then etat := 1; end if;
                when 1 =>
                    if SW_0 = '1' then etat := 2; end if;
                when 2 =>
                    if SW_0 = '0' then etat := 3; end if;
                when 3 =>
                    if SW_0 = '1' then etat := 4; end if;
                when 4 =>
                    if SW_0 = '0' then etat := 5; end if;
                when 5 =>
                    if SW_0 = '1' then
                        etat := 1;
                    else
                        etat := 0;
                    end if;
                when others =>
                    etat := 0;
            end case;

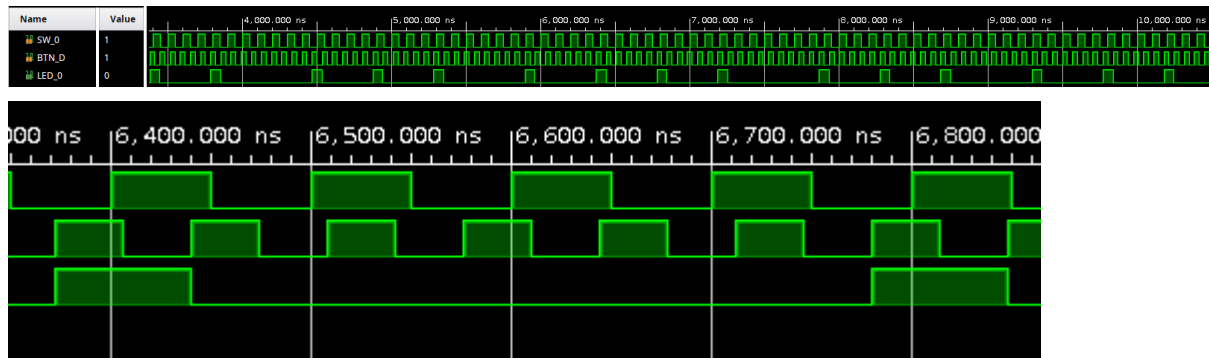
            if etat = 5 then
                LED_0 <= '1';
            else
                LED_0 <= '0';
            end if;

        end if;
    end process;
end architecture Behavioral;

```

Figure 4.2 : Code VHDL du detecteur de code assoupli

Commen pour l'exercice 3, on crée deux clocks binaires avec des périodes différentes pour les deux entrées, et on lance la simulation. On remarque que la LED_0 s'allume seulement une fois qu'on a bien eu des fronts montants du bouton poussoir BTN_D avec SW_0 sur 1, 1, 0, 1 puis 0. Cependant, comme on peut le remarquer sur la figure 4.4, les erreurs ne réinitialisent plus la machine. Ici, on a le code 110110 , et même avec le 1 en trop en gras, LES_0 finit par s'allumer.



Figures 4.3 - 4.4 : Simulation du détecteur de code assoupli

Dans le code VHDL, la variable parcourt 6 valeurs, de 0 à 5, donc théoriquement 3 bits suffiraient. Cependant, comme pour l'exercice 3, le synthetiseur privilégie un encodage à 6 bits. Le nombre d'état est donc le même que pour l'exercice précédent.

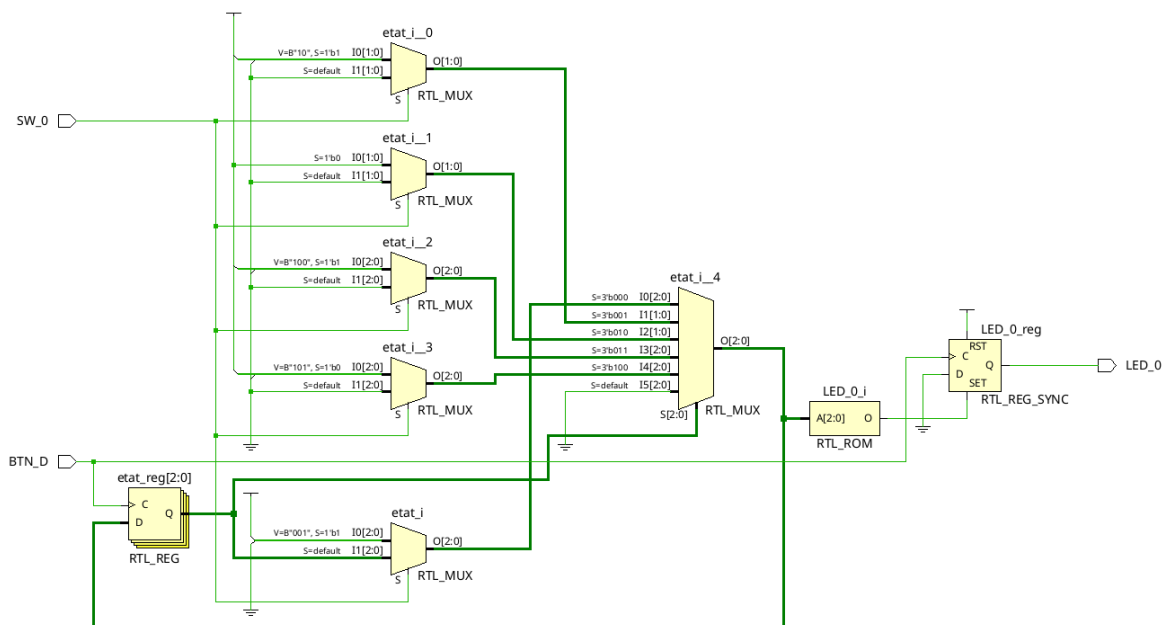


Figure 4.5 : Schéma du détecteur de code assoupli avant synthetisation

FSM_onehot_etat[0]_i_1						
I5	I4	I3	I2	I1	I0	O=I0 & I1 & !I5 + I2 & !I4 & I5 + !I2 & I3 8
0	0	0	0	0	0	1
0	0	0	0	0	1	1
0	0	0	0	1	0	0
0	0	0	0	1	1	1
0	0	0	1	0	0	1
0	0	0	1	0	1	0
0	0	0	1	1	0	1
0	0	0	1	1	1	1
0	0	1	0	0	0	1
0	0	1	0	0	1	0
0	0	1	0	1	0	1
0	0	1	0	1	1	1
0	0	1	1	0	0	1
0	0	1	1	0	1	0
0	0	1	1	1	0	1
0	0	1	1	1	1	1
0	1	0	0	0	0	1
0	1	0	0	0	1	0
0	1	0	0	1	0	1
0	1	0	0	1	1	1
0	1	0	1	0	0	1
0	1	0	1	0	1	0
0	1	0	1	1	0	1
0	1	0	1	1	1	1
0	1	1	0	0	0	1
0	1	1	0	0	1	0
0	1	1	0	1	0	1
0	1	1	0	1	1	1
0	1	1	1	0	0	1
0	1	1	1	0	1	0
0	1	1	1	1	0	1
0	1	1	1	1	1	1
1	0	0	0	0	0	0
1	0	0	0	0	1	1

FSM_onehot_etat[0]_i_1						
I5	I4	I3	I2	I1	I0	O=I0 & I1 & !I5 + I2 & !I4 & I5 + !I2 & I3 8
0	1	1	1	1	0	1
0	1	1	1	1	1	1
1	0	0	0	0	0	0
1	0	0	0	0	1	1
1	0	0	0	1	0	0
1	0	0	0	1	1	1
1	0	0	1	0	0	1
1	0	0	1	0	1	1
1	0	0	1	1	0	1
1	0	0	1	1	1	1
1	0	1	0	0	0	1
1	0	1	0	0	1	1
1	0	1	0	1	0	1
1	0	1	0	1	1	1
1	0	1	1	0	0	1
1	0	1	1	0	1	1
1	0	1	1	1	0	1
1	0	1	1	1	1	1
1	1	0	0	0	0	1
1	1	0	0	0	1	1
1	1	0	0	1	0	1
1	1	0	0	1	1	1
1	1	0	1	0	0	1
1	1	0	1	0	1	1
1	1	0	1	1	0	1
1	1	0	1	1	1	1
1	1	1	0	0	0	1
1	1	1	0	0	1	1
1	1	1	0	1	0	1
1	1	1	0	1	1	1
1	1	1	1	0	0	1
1	1	1	1	0	1	1
1	1	1	1	1	0	1
1	1	1	1	1	1	1

FSM_onehot_etat[1]_i_1			
I2	I1	I0	O=I0 & I1 + I1 & I2
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

FSM_onehot_etat[4]_i_1				
I3	I2	I1	I0	O=I0 & I1 + !I1 & I2 & I3
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

LED_0_i_2					
I4	I3	I2	I1	I0	O=!I0 & I1 & I2 & I3 + !I1 & I3 & !I4 + !I0 & I3 & !I1
0	0	0	0	0	0
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	0	1	0
0	0	1	1	0	1
0	0	1	1	1	0
0	1	0	0	0	1
0	1	0	0	1	1
0	1	0	1	0	1
0	1	0	1	1	0
0	1	1	0	0	1
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	0
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	0	1	0
1	1	0	1	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	0

Figure 4.7 - 4.11 : Tables de vérité du détecteur de code

L'équation de sortie est strictement identique à celle de l'exercice 3, étant donné que LED_0 ne dépend que de l'atteinte de l'état final. On se retrouve donc encore avec:

$$\text{LED_0} = \text{FSM_oneshot_etat}[5]$$

Pour ce modèle de détecteur de code, nous testons la tolérance aux fausses entrées en entrant délibérément des bits incorrect dans la séquence: Pour tester, nous avons choisi la séquence 111010, c'est à dire le code avec un 1 en trop au début. Ensuite, comme pour l'exercice 3, nous utilisons le switch SW_0 pour choisir une entrée et le bouton poussoir BTN_D comme horloge. On remarque que même avec cette erreur, dès le bit suivant est envoyé, la séquence reprend son cours et la led LED_0 finit par s'allumer.

2. Visualisez le schéma « Implémentation ».

1. Le nombre de bits d'état est-il le même que dans l'exercice 3 ?

Le nombre de bits d'état du second détecteur de code est identique à celui du premier. Le système comporte toujours 6 états distincts. L'encodage utilisé par le logiciel après l'implémentation suit la même logique pour les deux exercices et on se retrouve alors avec 6 bascules à état, FSM_oneshot_etat[0] jusque FSM_oneshot_etat[5].

2. L'équation de sortie est-elle plus simple ?

L'équation de sortie n'est donc pas plus simple, étant donné qu'elle est parfaitement identique; pour que l'alarme s'allume (c'est à dire que LED_0 s'allume), il faut que la machine soit sur le dernier état, c'est à dire q5. Ce n'est donc pas l'équation de sortie qui change, mais juste les équations de transition d'un état à l'autre.